

PHOTO MARKET

Sistema de Comercialización de Obras Fotográficas

Documentación de Ejecución y Pruebas

Universidad Icesi

Facultad de Ingeniería, Diseño y Ciencias Aplicadas

Ingeniería de Sistemas

Computación en Internet II

Nombre	Código
Luis Felipe Cadena Cortés	A00395967
Fabio Felipe Murillo Rivas	A00401131
Juan Esteban Cuellar	A00402548

Marzo 2026-1

Índice

Contenido

Índice.....	2
1. Introducción.....	4
1.1 Descripción del Proyecto	4
1.2 Tecnologías Utilizadas.....	4
2. Requisitos Previos.....	5
2.1 Software Requerido	5
2.2 Verificación de Instalaciones	5
3. Configuración del Entorno.....	6
3.1 Configuración de PostgreSQL	6
Paso 1: Iniciar el servicio de PostgreSQL	6
Paso 2: Crear la base de datos.....	6
Paso 3: Configurar credenciales	6
4. Instalación del Proyecto	7
4.1 Clonar el Repositorio	7
4.2 Estructura del Proyecto	7
4.3 Descargar Dependencias	7
5. Ejecución de la Aplicación.....	8
5.1 Iniciar el Backend	8
Salida Esperada.....	8
5.2 Verificar Inicialización de Datos	8
6. Ejecución de Pruebas	9
6.1 Pruebas Unitarias	9
Pruebas Implementadas	9
Salida Esperada.....	9
6.2 Reporte de Cobertura con JaCoCo	9
Visualizar el Reporte	9
Métricas de Cobertura.....	10
7. Verificación del Sistema	10
7.1 Verificación en PostgreSQL.....	10
Consultas de Verificación.....	10
7.2 Verificación desde el Navegador (Opcional).....	11

8. Solución de Problemas	11
8.1 Problemas Comunes	11
Error: Connection refused	11
Error: database photo_market does not exist	11
Error: password authentication failed	11
Error: Port 8080 already in use	11
Error: Tests failed	12
8.2 Reinicio Completo del Sistema	12
9. Anexos	12
9.1 Comandos Útiles de Maven.....	12

1. Introducción

Este documento proporciona instrucciones detalladas para ejecutar y probar el sistema Photo Market, un backend desarrollado con Spring Boot para la comercialización de obras fotográficas exclusivas.

1.1 Descripción del Proyecto

Photo Market es un sistema backend que implementa:

- Modelo de datos completo con 8 entidades JPA
- Sistema de autenticación con roles y permisos
- Gestión de usuarios, fotógrafos y compradores
- Catálogo de fotografías con control de exclusividad
- Registro de transacciones de venta
- Pruebas unitarias con JUnit, Mockito y JaCoCo

1.2 Tecnologías Utilizadas

Componente	Tecnología
Lenguaje	Java 17
Framework	Spring Boot 3.2.0
Persistencia	Spring Data JPA / Hibernate
Base de Datos	PostgreSQL 12+
Testing	JUnit 5 + Mockito
Cobertura	JaCoCo
Build	Maven 3.8+

2. Requisitos Previos

Antes de ejecutar el proyecto, asegúrese de tener instaladas las siguientes herramientas:

2.1 Software Requerido

Herramienta	Versión Mínima	Propósito
Java JDK	17+	Ejecutar aplicación
Maven	3.8+	Gestión de dependencias
PostgreSQL	12+	Base de datos
Git	2.0+	Control de versiones

Nota: Se recomienda utilizar un IDE como IntelliJ IDEA o Eclipse para facilitar el desarrollo y depuración.

2.2 Verificación de Instalaciones

Ejecute los siguientes comandos para verificar las versiones instaladas:

```
java -version
mvn -version
psql --version
git --version
```

3. Configuración del Entorno

3.1 Configuración de PostgreSQL

Paso 1: Iniciar el servicio de PostgreSQL

Windows:

1. Abrir 'Servicios' desde el menú de Windows
2. Buscar 'postgresql-x64-XX'
3. Click derecho > Iniciar

Linux/Mac:

```
sudo service postgresql start
```

Paso 2: Crear la base de datos

Conectarse a PostgreSQL:

```
psql -U postgres
```

Ejecutar el siguiente comando SQL:

```
CREATE DATABASE photo_market;
```

Verificar la creación:

```
\l
```

Debe aparecer **photo_market** en la lista de bases de datos.

Paso 3: Configurar credenciales

Importante: El sistema está configurado para usar las siguientes credenciales por defecto:

Parámetro	Valor
Usuario	postgres
Contraseña	12345
Puerto	5432
Base de datos	photo_market

Si necesita cambiar estas credenciales, modifique el archivo **src/main/resources/application.properties**

4. Instalación del Proyecto

4.1 Clonar el Repositorio

Ejecute el siguiente comando para obtener el código fuente:

```
git clone https://github.com/Computacion-2/proyecto-final-webones.git
```

Navegar al directorio del proyecto:

```
cd photo-market
```

4.2 Estructura del Proyecto

El proyecto tiene la siguiente estructura:

```
photo-market/
├── docs/                # Documentación
├── src/
│   ├── main/
│   │   ├── java/        # Código fuente
│   │   │   └── com/photomarket/
│   │   │       ├── entity/    # Entidades JPA
│   │   │       ├── repository/ # Repositorios
│   │   │       └── service/    # Servicios
│   │   └── resources/
│   │       ├── application.properties # Configuración
│   │       ├── schema.sql            # Esquema DB
│   │       └── data.sql              # Datos iniciales
│   └── test/                # Pruebas unitarias
├── pom.xml                 # Dependencias Maven
└── README.md              # Documentación general
```

4.3 Descargar Dependencias

Ejecute Maven para descargar todas las dependencias necesarias:

```
mvn clean install
```

Este comando descargará:

- Spring Boot y sus módulos
- PostgreSQL driver
- JUnit y Mockito
- JaCoCo
- Otras dependencias configuradas en pom.xml

Nota: Este proceso puede tomar varios minutos la primera vez.

5. Ejecución de la Aplicación

5.1 Iniciar el Backend

Desde la raíz del proyecto donde esta el pom.xml, ejecute:

```
mvn spring-boot:run
```

Salida Esperada

Si todo está configurado correctamente, verá una salida similar a:

```
.
/\ \ / _ _ ' _ _ _ _ ( ) _ _ _ _ \ \ \ \ \
( ( ) \ _ _ | ' _ | ' _ | ' _ \ / _ | \ \ \ \ \
\ \ / _ _ | | _ | | | | | | | ( _ | | ) ) ) )
' | _ _ | . _ | | | | | _ \ , | / / / /

...
HikariPool-1 - Starting...
HikariPool-1 - Start completed.
...
Tomcat started on port(s): 8080 (http)
Started PhotoMarketApplication in 5.432 seconds
```

5.2 Verificar Inicialización de Datos

Al iniciar, el sistema carga automáticamente:

Entidad	Cantidad
Permisos	5
Roles	3
Usuarios (1 admin, 15 fotógrafos, 39 compradores)	55
Categorías	6
Fotógrafos	15
Compradores	39
Fotografías (49 disponibles, 11 vendidas)	60
Ventas registradas	11

Total de registros iniciales: 204 registros

6. Ejecución de Pruebas

6.1 Pruebas Unitarias

Para ejecutar todas las pruebas unitarias:

```
mvn test
```

Pruebas Implementadas

El proyecto incluye pruebas para los siguientes servicios:

1. UserService

- createUser() - Crear usuario con rol válido
- createUser_WithoutRole() - Validar excepción sin rol
- getAllUsers() - Obtener lista de usuarios
- getUserById() - Buscar usuario por ID
- updateUser() - Actualizar información
- deleteUser() - Eliminar usuario

2. RoleService

- createRole() - Crear rol con permisos
- createRole_WithoutPermissions() - Validar excepción sin permisos
- getAllRoles() - Obtener lista de roles
- addPermissionToRole() - Agregar permiso
- removePermissionFromRole() - Quitar permiso

3. PermissionService

- createPermission() - Crear nuevo permiso
- getAllPermissions() - Obtener lista de permisos
- getPermissionById() - Buscar permiso por ID

Salida Esperada

Al ejecutar las pruebas, verá:

```
[INFO] -----  
[INFO]  T E S T S  
[INFO] -----  
[INFO] Running com.photomarket.service.UserServiceTest  
[INFO] Tests run: 34, Failures: 0, Errors: 0, Skipped: 0  
...
```

6.2 Reporte de Cobertura con JaCoCo

Para generar el reporte de cobertura:

```
mvn jacoco:report
```

Visualizar el Reporte

El reporte se genera en formato HTML. Para visualizarlo:

4. Navegar a la carpeta: target/site/jacoco/
5. Abrir el archivo: index.html
6. El navegador mostrará la cobertura detallada

Métricas de Cobertura

El reporte incluye:

- **Cobertura de instrucciones:** Porcentaje de líneas ejecutadas
- **Cobertura de ramas:** Porcentaje de decisiones evaluadas
- **Complejidad ciclomática:** Medida de complejidad del código
- **Cobertura de métodos:** Porcentaje de métodos probados
- **Cobertura de clases:** Porcentaje de clases cubiertas

Nota: El proyecto alcanza una cobertura superior al 80% en los servicios principales (UserService, RoleService, PermissionService).

7. Verificación del Sistema

7.1 Verificación en PostgreSQL

Conectarse a la base de datos:

```
psql -U postgres -d photo_market
```

Consultas de Verificación

1. Verificar tablas creadas:

```
\dt
```

Debe mostrar 9 tablas:

- permissions
- roles
- role_permissions
- users
- categories
- photographers
- buyers
- photographs
- sales

2. Contar usuarios:

```
SELECT COUNT(*) FROM users;
```

Resultado esperado: **55**

3. Contar fotografías:

```
SELECT COUNT(*) FROM photographs;
```

Resultado esperado: **60**

4. Ver fotografías disponibles:

```
SELECT COUNT(*) FROM photographs WHERE status = 'AVAILABLE';
```

Resultado esperado: **49**

5. Ver roles y permisos:

```
SELECT r.name AS rol, p.name AS permiso
FROM roles r
JOIN role_permissions rp ON r.id = rp.role_id
JOIN permissions p ON rp.permission_id = p.id
ORDER BY r.name;
```

7.2 Verificación desde el Navegador (Opcional)

Si se implementaron controllers, puede acceder a:

```
http://localhost:8080/api/users
```

Nota: El taller se enfoca en el backend, por lo que los endpoints REST son opcionales.

8. Solución de Problemas

8.1 Problemas Comunes

Error: Connection refused

Causa: PostgreSQL no está corriendo.

Solución:

```
sudo service postgresql start
```

Error: database photo_market does not exist

Causa: La base de datos no fue creada.

Solución:

```
CREATE DATABASE photo_market;
```

Error: password authentication failed

Causa: Credenciales incorrectas en application.properties.

Solución:

Verificar y corregir en `src/main/resources/application.properties`

```
spring.datasource.password=SU_PASSWORD
```

Error: Port 8080 already in use

Causa: Otra aplicación está usando el puerto 8080.

Solución:

Cambiar el puerto en application.properties:

```
server.port=8081
```

Error: Tests failed

Causa: Dependencias no actualizadas o código modificado.

Solución:

```
mvn clean install
```

8.2 Reinicio Completo del Sistema

Si enfrenta múltiples errores, realice un reinicio completo:

7. Detener la aplicación (Ctrl + C)

8. Eliminar la base de datos

```
DROP DATABASE photo_market;
```

9. Crear nuevamente la base de datos

```
CREATE DATABASE photo_market;
```

10. Limpiar el proyecto

```
mvn clean
```

11. Reinstalar dependencias

```
mvn clean install
```

12. Ejecutar la aplicación

```
mvn spring-boot:run
```

9. Anexos

9.1 Comandos Útiles de Maven

Comando	Descripción
mvn clean	Limpiar archivos compilados
mvn compile	Compilar el código
mvn test	Ejecutar pruebas unitarias
mvn package	Generar archivo JAR

<code>mvn spring-boot:run</code>	Ejecutar la aplicación
<code>mvn jacoco:report</code>	Generar reporte de cobertura

9.2 Comandos Útiles de PostgreSQL

Comando	Descripción
<code>\l</code>	Listar bases de datos
<code>\c database_name</code>	Conectar a una base de datos
<code>\dt</code>	Listar tablas
<code>\d table_name</code>	Describir estructura de tabla
<code>\q</code>	Salir de psql

9.3 Contacto y Soporte

Para consultas o soporte relacionado con este proyecto:

- **Repositorio GitHub:** [Repositorio](#)
- **Docente:** Alejandro Muñoz Bravo - Computación en Internet II

Fin del Documento

Universidad Icesi - Marzo 2026-1